

XIMOD Architect — XML Builder

User Manual

The XML installer, designed for your Mods.

Skyrim · Fallout · Starfield · Oblivion · Morrowind

Version 1.0.5

Cross-platform: Windows · Linux · macOS



Contents

1. Overview	4
1.1 What is XIMOD Architect?	4
1.2 The FOMOD format	4
1.3 Installation and requirements	4
Windows	5
Linux	5
macOS	5
1.4 File structure	5
2. Getting started	6
2.1 Launching the application	6
2.2 Interface overview	6
2.3 Creating a new project	6
2.4 Opening and saving a project	6
2.5 First launch	7
2.6 Free windows and keyboard navigation	7
3. The menus	8
3.1 File menu	8
3.2 Options menu	8
3.3 Help menu	9
4. Features in detail	10
4.1 The Mod Info tab	10
4.2 The Install Steps tab	10
Groups and selection types	10
Plugins and default types	11
Plugin files	11
Group / page destination	11
Condition flags	11
4.3 The Required Installs tab	12
4.4 The Conditional Installs tab	12
4.5 The "Settings" window	13
4.6 Pre- and post-save scripts	13
Example — pre-save script (Windows)	14
Example — post-save script (Windows)	14
Linux / macOS equivalent	14
4.7 The "About" window	14
4.8 Generating the FOMOD files	15
4.9 Translation editor	15

4.10 XML editor.....	17
4.11 Font management.....	17
4.12 Merging two FOMODs	19
4.13 Exporting the distribution archive	19
4.14 The "Properties" window	19
4.15 Command-line mode (CLI)	20
4.16 Checking for updates	20
5. FOMOD tag reference.....	21
5.1 The info.xml file	21
5.2 The ModuleConfig.xml file	21
Root and header	21
Files and folders	22
Steps and groups.....	22
Options (plugins).....	22
Dependencies and conditions	23
6. The ModuleConfig.xml file explained in detail.....	24
6.1 How the installer reads the file	24
6.2 The root and the header	24
6.3 Installing files	24
6.4 Steps, groups and options	25
6.5 An option's type and its behaviour	26
6.6 Flags, dependencies and conditional installs.....	26
7. ModuleConfig.xml examples.....	28
7.1 Minimal example — required files.....	28
7.2 Exclusive choice — SelectExactlyOne.....	28
7.3 Conditional install — flags.....	28
7.4 One page, several steps — SelectAny + flags.....	29
Appendices.....	31
Appendix A — The Config.ini file.....	31
Appendix B — Keyboard shortcuts	31
Appendix C — Glossary.....	32
Appendix D — Complete FOMOD example	34
D.1 info.xml.....	34
D.2 ModuleConfig.xml	34
Appendix E — License and credits.....	37

1. Overview

1.1 What is XIMOD Architect?

XIMOD Architect is a cross-platform tool for creating FOMOD installers for Bethesda game mods (Skyrim, Fallout, Starfield, Oblivion, Morrowind, and so on). It lets you visually build a complete installation wizard — steps, option groups, plugins, conditional files — and then automatically generate the standardized XML files that mod managers know how to interpret.

The application supports eleven games, defined in the `Categories.json` file: the three editions of Skyrim (Legendary, Special Edition and VR), Fallout 3, Fallout: New Vegas, Fallout 4 and Fallout 4 VR, Oblivion and Oblivion Remastered, Morrowind, as well as Starfield. Because this file is external and editable, the list can be extended without recompiling the application.

The software is a rewrite in the Rust language of the original FOMODCreationTool. This overhaul brings better performance, identical behaviour on Windows, Linux and macOS, and a modern interface. XIMOD Architect also stands out for its very advanced internationalization system: the interface can be translated into a large number of languages, with dynamic loading of the fonts suited to each writing system.

The main features are as follows:

- creation of complete FOMOD installation packages;
- multi-step installation wizards;
- plugin groups with different selection types (exactly one, at least one, any, and so on);
- conditional file installation based on the user's choices;
- dependency patterns that drive a plugin's type;
- a system of "flags" (condition flags) controlling the flow of the installation;
- custom scripts run before or after saving;
- a multilingual interface and dynamic fonts;
- a native, transparent splash screen.

1.2 The FOMOD format

FOMOD ("Fallout Mod") is the standard format for mod installers for Bethesda games. A FOMOD installer is based on two XML files gathered in a subfolder named `fomod`:

- `info.xml` — the mod metadata: name, author, version, website, description, category.
- `ModuleConfig.xml` — the installation configuration: steps, groups, plugins, files, conditions.

The files produced by XIMOD Architect are compatible with the main mod managers:

- Mod Organizer 2 (MO2);
- Vortex;
- Nexus Mod Manager (NMM).

Note — The name of the `fomod` folder as well as the names `info.xml` and `ModuleConfig.xml` are imposed by the specification and recognized as-is by mod managers. They must never be renamed.

1.3 Installation and requirements

System requirements (indicative):

Item	Recommendation
System	Windows 10 or later, Linux (recent glibc) or macOS 11 or later.
Display	Graphics card compatible with OpenGL 3.3 (egui's "glow" renderer).
RAM	A few hundred MB — the application is lightweight.
Disk space	A few tens of MB, depending on the Noto fonts and the flags provided in assets/.

Windows

Place in the same folder the executable `ximod-architect.exe`, the `assets` folder (data, locales, fonts, images, icons) and, optionally, the file `assets/images/splash.png` for the splash screen. The application icon is embedded in the executable at compile time. Then run `ximod-architect.exe`.

Linux

An installation script is provided in `packaging/linux`:

```
# System-wide installation (requires sudo)
sudo ./packaging/linux/install.sh

# Installation for the current user
./packaging/linux/install.sh --user
```

Building from source requires Rust 1.70 or later and, on Linux, the following development libraries: `libgtk-3-dev`, `libxcb-render0-dev`, `libxcb-shape0-dev`, `libxcb-xfixes0-dev`, `libspeechd-dev`, `libxkbcommon-dev`, `libssl-dev` and `libx11-dev`.

macOS

A script creates an application bundle (`.app`) ready to distribute:

```
./packaging/macos/create-bundle.sh
```

The "XIMOD Architect.app" bundle is generated in the `dist` folder. A disk image (`.dmg`) can then be produced with the `hdiutil` utility.

1.4 File structure

The file tree accompanying the executable is as follows:

<code>ximod-architect(.exe)</code>	→ executable
<code>Config.ini</code>	→ configuration (created on first launch)
<code>assets/</code>	
<code>data/</code>	
<code>Categories.json</code>	→ games and Nexus categories
<code>Languages.json</code>	→ 6,777 languages (endonym, ISO codes, font)
<code>Countries.json</code>	→ 244 countries (names, flag, official languages)
<code>CountryLanguages.json</code>	→ languages spoken by country
<code>locales/</code>	→ translations (one folder per language)
<code>eng/main.ftl</code>	
<code>fra/main.ftl</code>	
...	→ 32 languages provided
<code>fonts/</code>	→ Noto fonts (one per writing system)
<code>icons/</code>	→ application icons
<code>images/</code>	
<code>splash.png</code>	→ splash screen
<code>svg/</code>	→ country flags (244 SVG files)

2. Getting started

2.1 Launching the application

At startup, XIMOD Architect first displays a native, transparent splash screen, with a fade effect, whose duration is adjustable (two seconds by default; the screen can be disabled). The main window then opens, centred on the primary display. Its initial size comes from the configuration file (1280 × 800 by default) and cannot go below 800 × 600. The title bar shows the application name followed by its version number.

2.2 Interface overview

The main window is made up of three areas:

- **the menu bar** at the top: File, Options and Help (see chapter 3);
- **the tab area** in the centre, which brings together the four working panels: Mod Info, Install Steps, Required Installs and Conditional Installs (see chapter 4);
- **the status bar** at the bottom: on the left, the latest message (at startup: "Ready"); on the right, when a project has unsaved changes, a yellow dot followed by the text "Modified".

The appearance of the interface (dark, light or system theme, font size, language) is set in the Settings window described in § 4.5.

2.3 Creating a new project

Creating an installer generally follows the steps below:

- **New project** — File menu → New, then choose the root directory containing your mod's files ("Browse..." button in the Mod Info tab).
- **Fill in the information** — in the Mod Info tab, enter the mod name, author, version, game, category, website, header image and description.
- **Build the wizard** — in the Install Steps tab, add steps, then option groups, then plugins.
- **Associate the files** — for each plugin, add the files or folders to install, then set their destination in the table if needed (it is empty by default, which installs to the root of the game's Data folder).
- **Define the conditions** — use flags and conditional installs to adapt the installation to the user's choices.
- **Save** — File menu → Save (Ctrl+S): XIMOD Architect creates the fomod folder and writes info.xml and ModuleConfig.xml into it.

2.4 Opening and saving a project

Saving always generates the fomod subfolder under the root directory, then writes the two XML files into it in UTF-8. The Save command stays disabled as long as no root directory has been defined.

Two opening modes are offered. "Open folder..." asks for the root directory and loads the project from its fomod subfolder. "Open file..." asks directly for an XML file: the root directory is then inferred (the file is assumed to be located in a fomod subfolder). When reading, XIMOD Architect automatically recognizes the file encoding (UTF-8, with or without a byte order mark, as well as UTF-16), which makes it possible to reopen installers produced by other tools.

Each project opened successfully is added to the top of the recent files list, accessible through File → Recent and manageable in the Recent Files tab of the Settings.

2.5 First launch

On the very first launch, XIMOD Architect starts in English and automatically opens the Settings window, so that the user can choose their country and then their language.

This behaviour is driven by the `FirstStart` key in the `Config.ini` file: it is 0 as long as the initial configuration has not been saved, and switches to 1 as soon as "Save" is clicked for the first time. Subsequent launches then start directly in the chosen language.

If the user closes the window with "Cancel", the key stays at 0 and the window reopens at the next launch.

2.6 Free windows and keyboard navigation

Several tools open in **free (independent) windows**: the "Country selection" picker, the "Properties" window, the translation editor and the XML editor. Each one is freely movable and resizable, including on a second screen, and works alongside the main window.

Centring and persistence — the first time it is opened, a free window is centred on the XIMOD main window. If the user moves or resizes it, its position and size are remembered when it is closed (and when the application exits), then restored the next time it opens. These values are kept in `Config.ini`, under the `[WindowPositions]` section (see Appendix A). The main window itself remembers its size but not its position: it stays centred at every launch.

Closing with Esc — the Esc key closes the active free window as well as the main window's modal dialogs (Settings, About, confirmation, validation report, scripts); for a confirmation, Esc acts as "Cancel" and never runs the action. Two exceptions: the translation editor (to avoid closing it accidentally while typing) and the XIMOD main window do not close on Esc.

Keyboard navigation — the lists in the "Properties" window, the translation editor table and the flag grid of the country picker can be operated entirely from the keyboard: arrow keys (and the numeric keypad), Page Up/Page Down, Home/End, Enter and Tab/Shift+Tab. The details, specific to each window, are described in the relevant sections and summarised in Appendix B.

Clearing filters — every "Filter:" field has, to its right, a clear icon (red cross) that empties the filter with a single click.

3. The menus

The menu bar contains three menus: File, Options and Help. The shortcuts shown are also active globally (the Ctrl key corresponds to Cmd on macOS). Menus and shortcuts are ignored while a modal window is open.

3.1 File menu

Command	Shortcut	Description
New	Ctrl+N	Resets the project. If there are unsaved changes, confirmation is requested first.
Open folder...	Ctrl+O	Chooses a root directory and loads the project from its fomod subfolder.
Open file...	Ctrl+Shift+O	Chooses an XML file directly; the root directory is inferred.
Save	Ctrl+S	Generates fomod/info.xml and fomod/ModuleConfig.xml. Disabled as long as no root directory is defined.
Merge FOMOD...	—	Adds the steps, required files and conditional installs of another FOMOD to the current project. Disabled as long as no project is open. See § 4.12.
Export distribution archive...	—	Writes the FOMOD then compresses the whole root directory into a .zip archive ready to publish. Disabled as long as no root directory is defined. See § 4.13.
Recent	—	Submenu listing the most recently opened projects. Shows "(empty)" if the list is empty.
Quit	Ctrl+Q	Closes the application. The configuration is saved on exit.

3.2 Options menu

Command	Shortcut	Description
Settings	Ctrl+,	Opens the Settings window (country, language, theme, font, etc.). See § 4.5.
Pre-Save Script...	—	Edits the script run before saving. See § 4.6.
Post-Save Script...	—	Edits the script run after saving. See § 4.6.
Translation...	—	Opens the interface translation editor. See § 4.9.
XML Editor	—	Submenu giving access to info.xml and ModuleConfig.xml. See § 4.10.
Validate FOMOD	—	Checks the project and the conformity of ModuleConfig.xml to the ModConfig 5.0 schema, then shows a detailed report. See § 4.8.
Properties...	—	Opens the read-only explorer of the country / language database. See § 4.14.

3.3 Help menu

Command	Shortcut	Description
About	F1	Displays the "About" window (name, version, license). See § 4.7.

4. Features in detail

4.1 The Mod Info tab

This tab brings together the mod metadata and the choice of root directory.

Workspace — the "Root directory:" line shows the working folder (or "(Select a directory)" if it is not defined). The "Browse..." button lets you choose it.

The metadata fields are then as follows:

Field	Role
Mod Name	Displayed name of the installer (required).
Author	Name of the mod author.
Version	Version number (1.0.0 by default for a new project).
Game Name	Target game, chosen from a list drawn from Categories.json.
Category	Category of the mod. Depends on the chosen game; otherwise a default list is offered (Animation, Armour, Audio, Textures, Weapons, etc.).
Website	Address of the mod's page.
Header Image	Image shown at the top of the installer. "Browse..." and "Clear" buttons; a preview is displayed. The path is stored relative to the root directory.
Description	Presentation text of the mod (multiline area).

Note — Any change to a field marks the project as "Modified" in the status bar.

Nexus link — next to the "Game Name" selector, a "Nexus ↗" button opens, directly in the browser, the Nexus Mods page of the chosen game (built from its "slug"). It is active only if the selected game has a slug.

The "Game Name" and "Category" lists come from the external file `assets/data/Categories.json`, editable without recompilation. Each game is described there by an identifier, a name, a Nexus "slug" and its list of categories:

```
{
  "games": {
    "skyrimSpecialEdition": {
      "name": "The Elder Scrolls V: Skyrim Special Edition",
      "nexusSlug": "skyrimspecialedition",
      "categories": ["Animation", "Armour", "Audio", "..."]
    }
  }
}
```

4.2 The Install Steps tab

This is the heart of the installation wizard. It is based on a three-level hierarchy: Steps → Groups → Plugins. A row of tabs at the top represents the steps; the "+" button adds a step (named "Step N" by default). Below the step name (editable) and the "Delete step" button, the screen is split into two columns: on the left the groups and plugins, on the right the details of the selected plugin.

Groups and selection types

Each group has a name and a selection type that determines how many options the end user will be able to check. The five available types are:

Type	Meaning
SelectExactlyOne	The user must choose exactly one option.
SelectAtMostOne	At most one option (or none).
SelectAny	Any number of options (default value).
SelectAll	All options are selected.
SelectAtLeastOne	At least one option.

The "Add group" and "Delete group" buttons manage the list; a new group is of type SelectAny by default.

Reordering — the display order is adjusted without recreating everything: the step tabs carry "◀" / "▶" arrows to move the current step, and each group as well as each plugin has "▲" / "▼" arrows that move it up or down one notch in its list.

Plugins and default types

A plugin represents an option offered to the user. It has a name, a description, an image (preview, "Browse..." and "Clear" buttons) and a default type:

Type	Meaning
Optional	Optional option (default).
Required	Option that is always installed.
Recommended	Recommended but optional option.
NotUsable	Unusable option.
CouldBeUsable	Potentially usable option depending on conditions.

Plugin files

The "Source" section presents, as a four-column table (Type, Source, Destination, Priority), the files and folders installed by the plugin. The "Add file", "Add folder" and "Remove" buttons manage the list. The Destination column is editable: click a cell and type the path, relative to the game's Data folder, where the item must be installed. It is empty by default, which installs the file — or the contents of the folder — at the root of Data; this is the right choice when the source folder already reproduces the game's structure (for example a folder that itself contains meshes, textures, SKSE, etc.). Enter an explicit path only when the target differs, for example SKSE\Plugins\MyMod.ini. Files must be located under the root directory.

Group / page destination

To send several options to the same place without editing each file, use the bulk-destination controls. In a group's editor, the "Destination for the whole group" field and its "Apply to all plugins in this group" button set one install destination on every file of every plugin in that group at once. At the step level, the collapsible "Install destination (whole page)" panel does the same for every plugin on the page, across all its groups. A status message confirms how many files were updated. The button stays disabled until the plugins actually contain files, and you can still fine-tune any individual destination afterwards in the plugin's Source table.

Condition flags

The "Flag name:" section lets you associate flags with the plugin (name = value pairs). When a plugin is selected by the end user, its flags are set; they can then condition the visibility of a step, the type of a

plugin (dependency pattern) or the installation of conditional files. The "Add flag" and "Remove flag" buttons manage the list.

Autocompletion — the name and value fields of flags and dependencies offer, as you type, a menu of suggestions drawn from the elements already present in the project, which avoids typos that would silently break a match. For a file dependency, the value suggests the states Active / Inactive / Missing. The suggestions are not binding: a new value can still be entered freely.

Plugin dependencies. Below the flags, this section makes an option's type dynamic. The Default Type above applies when no pattern matches; each pattern gives the option another type when its conditions are met. Use New Pattern / Delete Pattern to manage the list; for the selected pattern choose the Pattern Operator (And / Or) and the Pattern Type to apply (Optional, Required, Recommended, NotUsable or CouldBeUsable), then add its conditions. A condition has a type (File or Flag), a File/Flag name (with autocompletion) and, for a file, a State (Active, Inactive or Missing) or, for a flag, its expected value. For example, set the Default Type to NotUsable and add a pattern of type Optional whose condition is the file DLCCoast.esm in state Active, so the option becomes selectable only when Far Harbor is installed.

Step visibility conditions. Each step can be shown conditionally. Under the step name, the collapsible Visibility Conditions section holds the step's visibility rule: set the Operator (And / Or) and add File or Flag conditions the same way. When it contains at least one condition, the step appears in the wizard only when they are satisfied — for example a patch step shown only when DLCCoast.esm is Active. Leave it empty to always show the step.

4.3 The Required Installs tab

This tab lists the files and folders installed systematically, regardless of the user's choices. It presents the same four-column table (Type, Source, Destination, Priority) — with the same editable Destination column — and the same "Add file", "Add folder" and "Remove" buttons as the files section of a plugin.

4.4 The Conditional Installs tab

This tab groups patterns: each pattern installs a set of files only when its conditions are met. A row of "Pattern N" tabs lets you add (" + ") or remove (" - ") a pattern.

For the selected pattern:

- **Operator** — And or Or: how the pattern's dependencies are combined.
- **Dependencies** — a list of conditions of type Flag ([Flag] name = value) or File ([File] name (state)). The "Add dependency" and "Remove dependency" buttons manage the list.
- **Files** — the four-column table of files installed when the conditions are satisfied.

Note — "plugin dependencies". A *plugin dependency* is simply a **File** dependency whose file is a game plugin (.esm/.esp). In the **name** field, enter the plugin's filename as it appears in the game (for example Fallout4.esm, DLCCoast.esm, or another mod's .esp), not the mod's display name. Choose the **state**: *Active* (plugin present and enabled — the usual choice), *Inactive* (present but disabled) or *Missing* (absent). A File dependency can be used in three places: to install files automatically (the **Conditional Installs** tab), to show or hide a wizard step (a step's *visible* condition), or to change an option's state — for example make it *Recommended* when a DLC is active, or *NotUsable* when a required plugin is missing.

4.5 The "Settings" window

Accessible through Options → Settings (Ctrl+,). It has two tabs: General and Recent Files. The settings are kept in Config.ini.

The **General** tab is organized into two columns.

Left column — the country flag and its name:

Item	Role
Flag	Clicking the image opens the country selector (see below).
Country name	Name of the country written in the selected language (endonym). Non-editable field, shown on two lines.

Right column — the interface settings:

Setting	Description
Language	Interface language, applied immediately. The field stays disabled as long as no country is chosen; it then offers the languages spoken in that country.
Theme	Dark, Light or System.
Font size	Size of the interface text.
Process line breaks	Interprets \n sequences in descriptions.
Max recent files	Number of entries kept.
Window Width / Height	Stored dimensions of the main window.

Country selector — clicking the flag opens the "Country selection" window, which presents the 244 flags as a grid, with vertical and horizontal scrolling. It is a free window, centred on the main window when it opens (see § 2.6). A "Filter:" field lets you narrow the list by French name, English name or country code; a clear icon (red cross) to the right of the field empties the filter with a single click. The country name appears under each flag.

The grid can also be navigated **from the keyboard**: the arrow keys (or the numeric keypad 8/2/4/6) move the selection frame, Page Up/Page Down scroll one screen, Home/End jump to the first and last flag, Tab and Shift+Tab move forward and back one flag, Enter confirms the highlighted country and Esc closes the window. Details are given in Appendix B.

The choice of country determines the list of languages offered: XIMOD relies on the CountryLanguages.json file, which lists all the languages spoken in each country — up to 844 for Papua New Guinea, hence the scrolling of the drop-down list.

4.6 Pre- and post-save scripts

XIMOD Architect can automatically run a script at save time: a pre-save script, launched just before the FOMOD files are written, and a post-save script, launched just after. They serve to automate the tasks surrounding mod generation — for example cleaning up old archives before saving, incrementing a build number, or automatically packaging the mod into an archive ready to publish once the FOMOD is written.

They are edited through Options → Pre-Save Script... and Options → Post-Save Script.... The window shows a help text, the list of available macros, an input area in a fixed-width font, and the "Save" and "Cancel" buttons. The contents of both scripts are kept in Config.ini (keys PreSaveScript and PostSaveScript), and are therefore remembered from one session to the next.

How it works: at save time, XIMOD first replaces the macros with their values, writes the result to a temporary file, then runs it with the system interpreter — a .bat command file launched by cmd on Windows, a .sh shell script launched by sh on Linux and macOS. The temporary file is deleted afterwards. The pre-save script runs before the XML files are written, the post-save script just after; a script left empty is simply ignored.

Before execution, the following macros are replaced in the script text:

Macro	Substituted value
\$MODNAME\$	Mod name.
\$MODAUTHOR\$	Author name.
\$MODVERSION\$	Mod version.
\$MODROOT\$	Path of the root directory.
\$DATE\$	Current date (YYYY-MM-DD).
\$TIME\$	Current time (HH:MM:SS).
\$RANDOM\$	Random number.

Example — pre-save script (Windows)

Delete the .rar archives already present at the root of the mod, to start from a clean folder before saving:

```
del "$MODROOT$\"*.rar" /q
```

Example — post-save script (Windows)

Once the FOMOD is written, automatically package the entire mod content into a .rar archive named after the mod name and version, using WinRAR:

```
d:
cd "$MODROOT$"
"C:\Program Files\WinRAR\rar.exe" a -r -ep1 "$MODNAME$_$MODVERSION$.rar" *
```

Line by line: d: switches to drive D:, cd "\$MODROOT\$" moves to the root of the mod, then WinRAR creates the archive (-r recursive, -ep1 so as not to reproduce the parent tree) named, for example, Dawnlight_1.2.0.rar.

Linux / macOS equivalent

On Linux and macOS, the script is interpreted as a shell script. The same post-save packaging is written, for example, with zip:

```
cd "$MODROOT$"
zip -r "$MODNAME$_$MODVERSION$.zip" .
```

Note — These scripts run real system commands with your user rights; a command such as del or rm is destructive. Test them on a throwaway project before using them on a real mod.

4.7 The "About" window

Opened through Help → About (F1), this window recalls the application name, its version number, a brief description ("A cross-platform tool for creating FOMOD installers for Bethesda game mods.") and the note "Licensed under MIT". An "OK" button closes it.

4.8 Generating the FOMOD files

On save, XIMOD Architect translates the project into two standardized XML files, written in UTF-8 in the fomod subfolder.

Before writing, the project is checked: if it contains anomalies (missing mod name, no step or required file, a step or group without a name, a group without a plugin), a window lists them all and offers "Save anyway" — saving is never blocked, which lets you deliberately save a work-in-progress project.

Schema validation — on top of this project check, there is a structural check of the generated ModuleConfig.xml against the ModConfig 5.0 schema, written in Rust (without an external XSD engine): element hierarchy and cardinality, mandatory attributes and enumeration values (selection and plugin types, file states, operators). Each anomaly is located by its line and column. It is accessible in three ways: the Options → Validate FOMOD command, which shows a full report; live in the XML editor as soon as the document is well-formed; and integrated into the save confirmation window.

info.xml contains the metadata, under a <fomod> root: the name (Name), the author (Author), the version (Version), the website (Website), the description (Description), the category (Groups) and the game (Game). The Game element is a XIMOD-specific extension, harmlessly ignored by mod managers, that serves to restore the correct list of categories when reloading.

ModuleConfig.xml describes the installation, under a <config> root: the module name, the header image, the required files (requiredInstallFiles), the steps (installSteps → group → plugin), and the conditional installs (conditionalFileInstalls). Each plugin describes its description, its image, its condition flags, its files and its type. Dependencies are combined by an And or Or operator.

4.9 Translation editor

Accessible through Options → Translation, this tool lets you translate the interface without editing the files by hand, and correct an existing translation.

Window header

Field	Role
Flag	Choice of country, which determines the list of translatable languages.
Country endonym name	Name of the country in the translated language, shown with the chosen font.
Language endonym name	Name of the language in that language itself (e.g. "Français").
Author	Name of the translator.
Displayed language	Reference language shown in column 2 (among those that already have a main.ftl file).
Language to translate	Target language, chosen from the languages of the country.
Font	Font used for this language. The "Browse..." button opens a file selector positioned on assets/fonts.
Google Fonts	Opens the fonts.google.com site in the browser.

Important constraint: the font must be installed in the assets/fonts folder. A font located elsewhere is rejected, because the program loads it via a path relative to that folder and would not find it at the next startup. If the font is missing, you therefore need to download it from Google Fonts and copy it there before selecting it.

Likewise, only a language that has an **ISO 639-3 code** can be translated: it is this code that names the translation folder and links the language to its font and its reference data. The save buttons are disabled otherwise.

Translation table — three columns:

Column 1	Column 2	Column 3
Key (non-editable)	Label in the displayed language	Translation input field

Token protection: the elements that must not be translated (macros such as `$MODNAME$`, variables such as `{ $num }`) never appear as-is in column 2 and are preserved automatically.

- Token at the beginning or end of the text: hidden, then reinserted on save.
- Token in the middle of the text: replaced by a compact marker `[[1]]`, `[[2]]`... to keep and reposition.

The application name (key `app-title`) is deliberately excluded and is not translatable.

Keyboard navigation — the table can be operated without the mouse (see Appendix B): the Up/Down arrows and Page Up/Page Down scroll the rows, Home/End jump to the first and last, and Enter places the cursor in the input field of the highlighted row. Once editing, Tab moves to the next field and Shift+Tab to the previous one, so translations can be entered one after another without leaving the keyboard. The number of rows moved by Page Up/Page Down adapts automatically to the window height.

Saving — the "Save" button writes the file `assets/locales/<language>/main.ftl`, preceded by a metadata block written as Fluent comments (the file therefore remains perfectly valid):

```
# XIMOD Architect - translation metadata
# @language = fra
# @font = Noto_Sans/static/NotoSans-Regular.ttf
# @langname = Français
# @author = Patrice
```

The header describes only the **language**: it contains no country information, because the same language is spoken in several countries (German in Germany as well as in Switzerland). The country is determined by the chosen flag, and XIMOD handles the country → languages mapping from `Countries.json` and `CountryLanguages.json`.

Filling in the endonym fields — when the country (flag) and the language are chosen, XIMOD fills in the two fields:

- **Country endonym name:** always resolved from `Countries.json` for the selected country; it therefore follows the flag (German shows "Deutschland" for Germany, "Schweiz" for Switzerland).
- **Language endonym name:** taken from the file header (`# @langname`) when it has one; failing that — an old file without a header, or a not-yet-translated language — it is taken from `Languages.json`.

The two fields remain editable: the user can make corrections there, which are propagated to the reference files on save.

Saving updates the **reference data**: the endonym and font of the language in `Languages.json`, and the country endonym in `Countries.json` (for the country / language pair). If the language did not yet appear there for this country, an entry is created.

Since the header no longer carries the country, XIMOD writes, alongside the `main.ftl`, a small tab-separated file **countryEndonyms.tsv**, which records the country endonym for the language, one line per country in the form "country code, language code, endonym". It is this file that makes it possible to recover the country endonym when only the language folder is available.

Sending the translation — the "Send..." button builds a ZIP archive containing the translation file (main.ftl, in its language folder) and, if it exists, the accompanying countryEndonyms.tsv file; XIMOD opens the archive folder then prepares a pre-filled email. The language-specific information (language, font, language endonym, author) travels in the header of the main.ftl; the country endonym travels in countryEndonyms.tsv. Neither Languages.json nor Countries.json is attached — only the codes actually used are transmitted, which respects the terms of use of the ISO 639-3 codes. The user attaches the file and sends it themselves: no data leaves the machine without an explicit action.

Note — *Only send the package when the translation is complete: any key left untranslated is saved with an empty value, which would then be displayed without text in the interface.*

4.10 XML editor

Accessible through Options → XML Editor → info.xml or ModuleConfig.xml, this tool intended for advanced users lets you view and directly modify the generated XML.

It opens in an **independent window**, freely movable and resizable, including on a second screen.

Two modes

- *Read-only* (default): the XML is shown with syntax highlighting and hanging indentation — lines that are too long wrap, aligning under the start of the line, in the manner of Notepad++. The main window remains usable.
- *Edit* ("Modify" button): the text becomes editable and the tabs of the main window are locked, so that a single source is authoritative.

Syntax highlighting: tags, attribute names, quoted values, comments, declaration and text are distinguished by colours suited to the light or dark theme.

Line numbering: a gutter shows the numbers; the continuation lines of a wrap remain blank.

Live validation: while typing, the XML is checked continuously. A green indicator confirms a well-formed document; in case of error, a red message specifies the line and column, and the faulty token is underlined.

Applying changes: the "Apply" button re-parses the text and updates the project. In case of error, the change is refused and the text remains editable. The "Cancel" button regenerates the text from the project.

4.11 Font management

XIMOD Architect ships with a set of Noto fonts covering all the writing systems of the listed languages, placed in assets/fonts.

These fonts are loaded **dynamically**: at any time, only those actually needed are installed, namely the font of the interface language and those of the languages of the selected country. The load stays low — a single font for most countries.

This mechanism guarantees the correct display of non-Latin scripts (Thai, Burmese, Amharic, Hmong, etc.), which would otherwise appear as empty squares. It applies both to the interface and to the "Properties" window and the translation editor: the fonts of every language displayed there are loaded so that no name or label appears as squares.

XIMOD Architect ships with **32 interface translations**, each covering the full set of strings, including non-Latin scripts: Japanese (jpn), Simplified Chinese (zho), Korean (kor) and Russian (rus). English (eng) and

French (fra) are the reference translations, maintained by hand; the others come from contributions that native speakers can review through the translation editor.

The font associated with each language is indicated in Languages.json and can be changed from the translation editor.

4.12 Merging two FOMODs

The File → Merge FOMOD... command lets you combine two installers by joining their installation content. The currently open project acts as the recipient; it must therefore have been created or opened beforehand, otherwise the command stays disabled.

A click opens a file selector: choose the ModuleConfig.xml file of the FOMOD to integrate (the donor). Its steps, required files and conditional installs are then added to the end of those of the recipient project. The metadata of the recipient project — name, author, version, category, header image — is preserved: only the installation content is imported.

After the merge, the project is marked as modified: remember to save it (Ctrl+S) to write the result to the FOMOD files. The merge can be repeated to integrate several FOMODs one after another.

Note — Only the donor's ModuleConfig.xml file is read; its own metadata (info.xml) is not imported. After merging, check that priorities and conditions remain consistent across the combined steps.

4.13 Exporting the distribution archive

The File → Export distribution archive... command produces, in a single operation, the archive ready to be uploaded (Nexus Mods or any other mod manager). XIMOD first writes the FOMOD (fomod/info.xml and fomod/ModuleConfig.xml) into the root directory, then compresses the whole folder in ZIP format. The command is disabled as long as no root directory is defined.

A selector offers a default name of the form "<name>-<version>.zip" (sanitized of forbidden characters); you can change it before saving. The status bar then indicates the number of files written and the path of the archive.

Archive layout — the fomod/ subfolder and the mod files are at the root of the archive, exactly as mod managers expect. Internal paths use the forward slash "/" (cross-platform compatibility).

Note — Junk files (.git, .DS_Store, Thumbs.db, desktop.ini...) and the archive itself are automatically excluded. Make sure, therefore, that all the mod files are indeed under the root directory before exporting.

4.14 The "Properties" window

Opened through Options → Properties..., this independent window — movable and resizable, including on a second screen — offers a read-only explorer of the embedded country / language database. The database cannot be modified here; it is a consultation tool.

Countries tab: for a chosen country, the window shows its flag, its French and English names, its official languages (with endonym, font and ISO 639-3 / 639-1 codes) and the list of all the languages spoken there.

Languages tab: for a chosen language, the window shows its ISO codes, its font, and performs the reverse lookup of the countries where this language is spoken.

Keyboard navigation — both the country list and the language list can be operated from the keyboard (see Appendix B): Up/Down arrows, Page Up/Page Down (whose step adjusts to the window height), Home/End and Enter to select the highlighted item. Tab and Shift+Tab move the selection within the list without ever landing on the flag. Each "Filter:" field has a clear icon (red cross) to its right, and the window closes with Esc.

4.15 Command-line mode (CLI)

Launched with arguments, XIMOD Architect opens no window: it runs the requested command then terminates with an exit code (0 on success, non-zero on failure or validation anomaly). This mode makes it possible to automate the rebuilding and packaging of one or more FOMODs in a script or a continuous-integration pipeline.

Note — On Windows, since the executable is compiled as a "windows" application, it attaches to the console that launched it (via `AttachConsole`) to write its messages there. Launch it from a command prompt (`cmd` or `PowerShell`) to see the output; double-clicking a CLI command would display nothing.

The available commands are:

Command	Effect
<code>ximod-architect validate <root></code>	Validates the FOMOD of the directory (project checks and conformity to the ModConfig 5.0 schema); each anomaly is located. Non-zero exit code if an anomaly is found.
<code>ximod-architect build <root></code>	(Re)writes <code>fomod/info.xml</code> and <code>fomod/ModuleConfig.xml</code> from the existing project.
<code>ximod-architect package <root> [-o FILE]</code>	Writes the FOMOD then builds the .zip distribution archive. The <code>-o</code> option lets you specify the output path; failing that, the archive is created in the current directory.
<code>ximod-architect help</code>	Displays the help (list of commands).
<code>ximod-architect version</code>	Displays the version number.

Here, `<root>` denotes the root directory of the mod (the folder that contains — or will contain — the `fomod` subfolder).

4.16 Checking for updates

By default, XIMOD Architect checks once a day, when it starts, whether a newer version is available. If one is, a discreet banner appears below the menu bar with links to download it (Nexus and GitHub); you can also check at any time from `Help` → `Check for updates`. This can be turned off in the `Settings` window ("Check for updates on startup").

Privacy: this check is the only network request the application makes. It sends no personal data — only a single secure (HTTPS) request to the public GitHub releases page to read the latest version number. XIMOD never downloads or installs anything by itself; you always choose whether to follow the download link.

5. FOMOD tag reference

This section describes the tags actually produced by XIMOD Architect. The two files are written in UTF-8 (with a byte order mark) in the mod's fomod subfolder: info.xml is indented in four-space steps, ModuleConfig.xml with a tab. When reading, the encoding is detected automatically thanks to the byte order mark, which makes it possible to open both UTF-8 and UTF-16 (LE or BE) files produced by other tools.

5.1 The info.xml file

The metadata, under a <fomod> root. The elements are written in the order of the table below; those marked "if not empty" are omitted when the corresponding field is left empty.

Tag	Content	Written
<Name>	Mod name.	Always
<Author>	Author.	If not empty
<Version>	Version.	If not empty
<Website>	URL of the mod's page.	If not empty
<Description>	Description.	If not empty
<Groups>	Category of the mod.	Always
<Game>	Identifier of the selected game. XIMOD-specific extension (outside the FOMOD specification), ignored by mod managers.	If not empty

Complete example:

```
<?xml version="1.0" encoding="utf-8"?>
<fomod>
  <Name>My Awesome Mod</Name>
  <Author>Patrice</Author>
  <Version>1.0.0</Version>
  <Website>https://www.nexusmods.com/skyrimspedition/mods/12345</Website>
  <Description>A sample mod for the manual.</Description>
  <Groups>Player homes</Groups>
  <Game>skyrimSpecialEdition</Game>
</fomod>
```

5.2 The ModuleConfig.xml file

The installation logic, under a <config> root that references the ModConfig5.0.xsd schema. The blocks appear in the following order: module name, header image, required files, install steps, conditional installs.

Root and header

Tag / Attribute	Role
<config>	Root. Carries the xmlns:xsi and xsi:noNamespaceSchemaLocation attributes (ModConfig5.0.xsd schema).
<moduleName>	Module name, shown at the top of the wizard.
<moduleImage path="..." />	Header image (empty tag, written only if an image is defined).

Files and folders

Tag / Attribute	Role
<code><file source="..." destination="..." priority="..." /></code>	Copy of a file. source = path in the archive (relative to the root); destination = target under Data; priority = installation order.
<code><folder source="..." destination="..." priority="..." /></code>	Copy of a whole folder (same attributes).
<code><requiredInstallFiles></code>	Container of the files installed systematically.
<code><files></code>	Container of the files tied to an option or a conditional pattern.

Steps and groups

Tag / Attribute	Role
<code><installSteps order="Explicit"></code>	Container of the steps (the defined order is preserved).
<code><installStep name="..."></code>	A page of the wizard.
<code><visible></code>	Visibility conditions of the step (contains <code><dependencies></code>).
<code><optionalFileGroups></code>	Container of the option groups of the step.
<code><group name="..." type="..."></code>	A group; type = selection rule (see below).
<code><plugins order="Explicit"></code>	Container of the options of the group.

Values of the type attribute of a `<group>`:

type	Constraint
SelectExactlyOne	Exactly one option.
SelectAtMostOne	At most one option.
SelectAny	Any number.
SelectAll	All, enforced.
SelectAtLeastOne	At least one.

Options (plugins)

Tag / Attribute	Role
<code><plugin name="..."></code>	An option offered to the user.
<code><description></code>	Descriptive text of the option (always written).
<code><image path="..." /></code>	Image illustrating the option (if defined).
<code><conditionFlags></code>	Container of the flags set if the option is chosen.
<code><flag name="...">value</flag></code>	A flag: name in the attribute, value in the content.
<code><typeDescriptor></code>	Describes the type of the option (simple or conditional).
<code><type name="..." /></code>	Simple type (when no dependency pattern is defined).
<code><dependencyType></code>	Dynamic type (when patterns exist).
<code><defaultType name="..." /></code>	Default type applied if no pattern applies.

Values of a `<type>` / `<defaultType>`:

Type	Effect in the wizard
Optional	Option unchecked, freely activatable.
Required	Mandatory option, cannot be deselected.
Recommended	Option checked by default, can be deselected.
NotUsable	Option disabled (cannot be selected).
CouldBeUsable	Usable, but flagged as potentially risky.

Dependencies and conditions

Tag / Attribute	Role
<code><dependencies operator="..."></code>	Group of conditions; operator = And or Or.
<code><fileDependency file="..." state="..."/></code>	Condition on a file; state = Active, Inactive or Missing.
<code><flagDependency flag="..." value="..."/></code>	Condition on a flag and its expected value.
<code><conditionalFileInstalls></code>	Section of the conditional installs.
<code><patterns> / <pattern></code>	List of patterns; each pattern associates <code><dependencies></code> with <code><files></code> (or, within a <code>typeDescriptor</code> , with a <code><type></code>).

6. The ModuleConfig.xml file explained in detail

Chapter 5 presented the tags as reference tables. This chapter goes through them one by one, in plain language, to explain what each one does, what options it accepts, and above all what concrete effect these options have on the installation wizard that the mod's user will see. No prior knowledge of XML is required: XIMOD Architect writes this file for you; understanding it only helps you design more reliable installers.

A few words of vocabulary. An XML file is made of "tags" enclosed in angle brackets. An opening tag `<step>` goes together with a closing tag `</step>`; whatever is written between the two belongs to that tag. A tag can carry "attributes", that is, settings written in the form `name="value"`. Thus `<group name="Resolution" type="SelectExactlyOne">` is a "group" tag with two attributes. Finally, a tag with no content closes on itself, like `<file ... />` (note the trailing `/`).

6.1 How the installer reads the file

When the user installs your mod, their manager (Vortex, MO2, NMM) reads ModuleConfig.xml from top to bottom and builds a wizard: a series of pages (the steps), each offering groups of options to check. At the end, the manager gathers all the files matching the chosen options and copies them into the game. Three ideas to keep in mind:

- **order matters**: the steps are displayed in the order they appear in the file;
- **checking installs nothing right away**: checking an option only records a choice; the actual copying of files happens at the very end;
- **conditions can modulate everything**: hiding a step, changing the state of an option or triggering the installation of additional files — this is the role of flags and dependencies (§ 6.6).

6.2 The root and the header

`<config>` — the root. The whole configuration fits inside a single `<config>...</config>` tag. It carries two technical attributes (`xmlns:xsi` and `xsi:noNamespaceSchemaLocation`) that designate the validation schema (ModConfig 5.0). You never have to touch them: XIMOD always writes them identically.

`<moduleName>` — the mod name, shown at the top of the wizard. It is the one entered in the Mod Info tab.

`<moduleImage path="..." />` — the banner image shown at the top of the wizard. The `path` attribute indicates its location, relative to the root of the mod (e.g. `fomod\header.png`). The tag is empty (it closes on itself). Without an image, it is simply absent.

6.3 Installing files

Two tags describe what will be copied into the game: `<file>` copies a single file, `<folder>` copies a whole folder with all its content. Both accept the same three attributes:

Attribute	Role	Example
source	Path of the element in your mod's archive, relative to the root.	tex\2k
destination	Where the element must land, relative to the game's Data folder.	textures
priority	Installation order in case of conflict (see below).	0

The priority attribute and its impact. It is a number that sets the copy order when two options install a file in the same place. The rule: low priorities are installed first, high ones afterwards — and since the last written wins, a higher priority "overwrites" a lower one. Example: if a base option installs a texture at `priority="0"` and a "high resolution" option installs the same texture at `priority="1"`, it is the high-resolution version that remains, regardless of the display order of the options. Leave 0 everywhere as long as you have no deliberate conflict to arbitrate.

Note — *The destination is empty by default, which installs the item at the root of the game's Data folder. Click the Destination cell to enter an explicit path (relative to Data) whenever the item must go elsewhere. An empty destination is the correct choice when the source folder already mirrors the game's structure.*

These tags appear in three places: in `<requiredInstallFiles>` (files always installed), in the `<files>` of an option (installed if the option is chosen), and in the `<files>` of a conditional pattern (installed if conditions are met).

6.4 Steps, groups and options

This is the heart of the wizard. The hierarchy is simple: a step contains groups, a group contains options.

`<installSteps order="Explicit">` gathers all the steps; `order="Explicit"` means "respect the order I wrote". Each page of the wizard is an `<installStep name="...">`, whose name attribute is the title shown at the top of the page.

`<visible>` — show the step conditionally. A step can be shown only if certain conditions are true; you place them in a `<visible>` tag (see § 6.6). Without `<visible>`, the step is always displayed. Example: show an "advanced settings" step only if the user checked "expert installation" earlier.

`<optionalFileGroups>` contains the groups of the step. Each `<group name="..." type="...">` has a name (section title) and, above all, a type that sets the selection rule — that is, what the user is allowed to check:

type	What the user sees and can do
SelectExactlyOne	Radio buttons: they must choose one option, and only one. Neither zero nor two. Ideal for exclusive variants (a single texture resolution).
SelectAtMostOne	As above, but they can also choose nothing: an "option or nothing".
SelectAny	Independent checkboxes: they check whatever they want, from none to all. The most common case, for stackable modules.
SelectAtLeastOne	Checkboxes, but they must check at least one to continue. Prevents leaving empty-handed.
SelectAll	All options are checked automatically and locked: the user cannot uncheck them. Used to present mandatory files while describing them.

In a group, `<plugins order="Explicit">` contains the options, each being a `<plugin name="...">` (the word "plugin" here means an option of the wizard, not an .esp file). Its name is the label the user checks. An option is made up of:

- `<description>`: the text shown on the right when the option is selected — describe its effect clearly there;
- `<image path="...">`: an illustration shown with the description (optional);
- `<conditionFlags>`: the flags set when the option is chosen (§ 6.6);
- `<files>`: the files installed if the option is chosen;

- `<typeDescriptor>`: the default state of the option (§ 6.5).

6.5 An option's type and its behaviour

Each option has a "type" that determines its default state in the wizard: checked or not, editable or not. It is described by `<typeDescriptor>`. In the simple case, it contains a single `<type name="..." />`. The five values and their effect:

type	Effect in the wizard
Optional	Unchecked by default, freely checkable and uncheckable. The common case.
Recommended	Checked by default, but the user can uncheck it. For an advised choice.
Required	Checked and locked: cannot be unchecked. For an essential component.
NotUsable	Unchecked and locked: cannot be checked. Indicates an option unavailable in the current context.
CouldBeUsable	Checkable, but flagged as potentially problematic (often with a warning).

Dynamic types: `<dependencyType>`. An option's type can also change depending on conditions. Instead of a fixed `<type>`, you write a `<dependencyType>` that contains a `<defaultType name="..." />` (the default type, when no condition is met) and a list of `<pattern>`: each pattern associates conditions with a `<type>` to apply if they are true.

Example: an "ENB Ultra preset" option can be Optional by default, become Recommended if the user enabled dynamic candles, and become NotUsable if a required file is missing on disk. It is this mechanism that makes an installer "smart". A complete example appears in Appendix D.

6.6 Flags, dependencies and conditional installs

These three notions form the FOMOD condition system — what allows an installer to react to the user's choices.

Flags — `<conditionFlags>` and `<flag>`. A flag is a small variable that you set when an option is chosen. An option's `<conditionFlags>` tag contains one or more `<flag name="...">value</flag>`: name is the flag name, and the text between the tags is its value. Thus `<flag name="res">4K</flag>` sets the "res" flag to "4K" as soon as the option is checked. A flag has no visible effect on its own: it serves as memory, reused later as a condition elsewhere.

Dependencies — `<dependencies operator="...">`. A dependency is a condition to check. They are grouped in `<dependencies>`, whose operator attribute says how to combine them: And (all conditions must be true) or Or (a single one is enough). Two kinds of conditions exist:

Condition	What it tests
<code><flagDependency flag="..." value="..." /></code>	True if the named flag has exactly the indicated value. This is how you react to choices made earlier (e.g.: the "res" flag equals "4K").
<code><fileDependency file="..." state="..." /></code>	True depending on the state of a file already present, or not, in the user's game.

The state attribute of a file dependency takes three values:

state	Meaning
Active	The file is present and enabled in the game.
Inactive	The file is present but disabled.
Missing	The file is absent.

These dependencies serve three things: the visibility of a step (in `<visible>`), the dynamic type of an option (in a `<pattern>` of `<dependencyType>`), and the conditional installs below.

Conditional installs — `<conditionalFileInstalls>`. This section, placed after the steps, installs files only when conditions are met, without the user having to check anything directly. It contains `<pattern>` elements, each associating `<dependencies>` with a `<files>`. If the pattern's conditions are true at the end of the wizard, its files are installed.

Difference with step visibility: `<visible>` decides whether a page is shown, whereas `<conditionalFileInstalls>` decides whether files are installed, behind the scenes. Typical case: automatically install a compatibility patch only if the user checked two particular options (whose flags are then both set).

Note — A flag must first be set by an option (via `<conditionFlags>`) before it can be tested by a dependency. So define your flags in the steps, then test them in the conditional patterns.

7. ModuleConfig.xml examples

The examples below illustrate the most common cases. They correspond to the XML actually produced by XIMOD Architect (indentation simplified for readability).

7.1 Minimal example — required files

A mod with no choices: all the files are installed directly.

```
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
  <moduleName>My Simple Mod</moduleName>
  <requiredInstallFiles>
    <file source="MyMod.esp" destination="MyMod.esp" priority="0"/>
    <folder source="textures" destination="textures" priority="0"/>
  </requiredInstallFiles>
</config>
```

7.2 Exclusive choice — SelectExactlyOne

A step offering the choice of a single texture resolution; the 2K option is recommended by default.

```
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
  <moduleName>HD Textures</moduleName>
  <moduleImage path="fomod\header.png"/>
  <installSteps order="Explicit">
    <installStep name="Texture resolution">
      <optionalFileGroups>
        <group name="Resolution" type="SelectExactlyOne">
          <plugins order="Explicit">
            <plugin name="2K">
              <description>2K textures (balanced).</description>
              <files>
                <folder source="2k" destination="textures" priority="0"/>
              </files>
              <typeDescriptor>
                <type name="Recommended"/>
              </typeDescriptor>
            </plugin>
            <plugin name="4K">
              <description>4K textures (high quality).</description>
              <files>
                <folder source="4k" destination="textures" priority="0"/>
              </files>
              <typeDescriptor>
                <type name="Optional"/>
              </typeDescriptor>
            </plugin>
          </plugins>
        </group>
      </optionalFileGroups>
    </installStep>
  </installSteps>
</config>
```

7.3 Conditional install — flags

An option sets a flag when it is checked; a file is installed only if that flag is active.

```

<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
  <moduleName>Conditional patch</moduleName>
  <installSteps order="Explicit">
    <installStep name="Options">
      <optionalFileGroups>
        <group name="Modules" type="SelectAny">
          <plugins order="Explicit">
            <plugin name="Module A">
              <description>Enables module A.</description>
              <conditionFlags>
                <flag name="ModuleA">On</flag>
              </conditionFlags>
              <files>
                <folder source="A" destination="meshes" priority="0"/>
              </files>
              <typeDescriptor>
                <type name="Optional"/>
              </typeDescriptor>
            </plugin>
          </plugins>
        </group>
      </optionalFileGroups>
    </installStep>
  </installSteps>
  <conditionalFileInstalls>
    <patterns>
      <pattern>
        <dependencies operator="And">
          <flagDependency flag="ModuleA" value="On"/>
        </dependencies>
        <files>
          <file source="patchA.esp" destination="patchA.esp" priority="0"/>
        </files>
      </pattern>
    </patterns>
  </conditionalFileInstalls>
</config>

```

7.4 One page, several steps — SelectAny + flags

A single multiple-choice step can open a different step for each option the player ticks: steps are not tied to options. Each option sets a condition flag, and each target step carries a visibility condition on that flag. Ticking A and B then shows steps A and B; ticking A, B and C shows three steps.

```

<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
  <moduleName>Modular Add-ons</moduleName>
  <installSteps order="Explicit">
    <!-- Step 1: the choices. Each ticked option sets its own flag. -->
    <installStep name="Choose modules">
      <optionalFileGroups>
        <group name="Modules" type="SelectAny">
          <plugins order="Explicit">
            <plugin name="Module A">
              <description>Enable module A.</description>
              <conditionFlags>
                <flag name="want_A">on</flag>
              </conditionFlags>
              <typeDescriptor><type name="Optional"/></typeDescriptor>
            </plugin>
            <plugin name="Module B">
              <description>Enable module B.</description>
              <conditionFlags>
                <flag name="want_B">on</flag>
              </conditionFlags>
              <typeDescriptor><type name="Optional"/></typeDescriptor>
            </plugin>
            <!-- Modules C and D follow the same pattern (want_C, want_D). -->
          </plugins>
        </group>
      </optionalFileGroups>
    </installStep>
    <!-- Step A: shown only when want_A is on. -->
    <installStep name="Module A options">
      <visible>
        <flagDependency flag="want_A" value="on"/>
      </visible>
      <optionalFileGroups>
        <group name="Module A" type="SelectAny">
          <plugins order="Explicit">
            <plugin name="A - extra textures">
              <description>Optional high-res textures for A.</description>
              <files>
                <folder source="a_textures" destination="textures" priority="0"/>
              </files>
              <typeDescriptor><type name="Optional"/></typeDescriptor>
            </plugin>
          </plugins>
        </group>
      </optionalFileGroups>
    </installStep>
    <!-- Step B: shown only when want_B is on. -->
    <installStep name="Module B options">
      <visible>
        <flagDependency flag="want_B" value="on"/>
      </visible>
      <optionalFileGroups>
        <group name="Module B" type="SelectAny">
          <plugins order="Explicit">
            <plugin name="B - extra meshes">
              <description>Optional meshes for B.</description>
              <files>
                <folder source="b_meshes" destination="meshes" priority="0"/>
              </files>
              <typeDescriptor><type name="Optional"/></typeDescriptor>
            </plugin>
          </plugins>
        </group>
      </optionalFileGroups>
    </installStep>
    <!-- Steps for C and D have exactly the same shape, each guarded by its
      own <visible><flagDependency flag="want_C | want_D" value="on"/>. -->
  </installSteps>
</config>

```

Appendices

Appendix A — The Config.ini file

The configuration is saved in the Config.ini file, located next to the executable. Here is a complete example:

```
; XIMOD Architect Configuration
[General]
Locale=fra
Country=FRA
FirstStart=1
Theme=Dark
FontSize=14
ReplaceNewlines=1
MaxRecentFiles=10
SplashScreenSeconds=2

[Window]
WindowWidth=1280
WindowHeight=800

[WindowPositions]
WinPos_ximod_translation=120,80
WinSize_ximod_translation=1100,700
WinPos_ximod_preview=200,140
WinSize_ximod_preview=900,650

[Scripts]
PreSaveScript=
PostSaveScript=

[RecentFiles]
RecentFile0=C:\Mods\MyMod
```

Two keys are added to the previous ones: Country (ISO 3166-1 alpha-3 code of the chosen country) and FirstStart (see § 2.5).

The [WindowPositions] section remembers the geometry of the free windows (see § 2.6). For each window, WinPos_<id> stores its position (x,y) and WinSize_<id> its size (width,height). These entries are only created once the user has actually moved or resized the window; without them, the window opens centred on the main window. The size of the main window is kept in [Window] (WindowWidth / WindowHeight), but its position is not.

Appendix B — Keyboard shortcuts

On macOS, the Ctrl key corresponds to Cmd.

Global shortcuts

Shortcut	Action
Ctrl+N	New project.
Ctrl+O	Open a folder.
Ctrl+Shift+O	Open a file.
Ctrl+S	Save (generate the FOMOD).
Ctrl+,	Open the Settings.
Ctrl+Q	Quit the application.

Shortcut	Action
F1	Show the "About" window.

Windows and dialogs

Shortcut	Action
Esc	Closes the active free window (Country selection, Properties, XML editor) or the modal dialog (Settings, About, confirmation, validation report, scripts). On a confirmation, acts as "Cancel". No effect on the translation editor or the main window.

List navigation — Properties and translation editor

Key	Action
↑ / ↓	Moves the selection one row up / down.
Page Up / Page Down	Scrolls one screen; the step adjusts to the window height.
Home / End	Jumps to the first / last row (the last row is brought fully into view).
Enter	Properties: selects the highlighted item. Translation editor: places the cursor in the row's input field.
Tab / Shift+Tab	Properties: next / previous row (never on the flag). Translation editor (while editing): next / previous translation field.

Flag grid — "Country selection" window

Key	Action
↑ ↓ ← → (or 8 2 4 6)	Moves the selection frame within the grid.
Page Up / Page Down	Scrolls one screen of flags.
Home / End	Jumps to the first / last flag (the last row is brought fully into view).
Tab / Shift+Tab	Next / previous flag.
Enter	Confirms the highlighted country.
Esc	Closes the window.

Appendix C — Glossary

Term	Definition
FOMOD	Standard mod installer format for Bethesda games, based on info.xml and ModuleConfig.xml.
Step	Page of the installation wizard, containing one or more groups.
Group	Set of options (plugins) sharing a selection type.
Plugin	Option offered to the user, associated with files to install.
Flag	Condition flag (name = value pair) used to drive the installation.
Endonym	Name of a country or language as it is written in that language.
ISO 639-3	Three-letter language code naming the translation folders (e.g. fra, eng).
ISO 3166-1 alpha-3	Three-letter country code (e.g. FRA, USA).

Term	Definition
Root directory	Working folder containing the mod files and the generated fomod subfolder.

Appendix D — Complete FOMOD example

This appendix presents a fictional but complete project — "Dawnlight", a lighting mod for Skyrim Special Edition — that puts into practice the full range of XIMOD Architect's possibilities: required files, multiple steps, a step visibility condition, the five group types, the five option types, condition flags, dynamic option type (dependency patterns) and conditional installs. The two files below form a single project and are reproduced as XIMOD generates them (simplified indentation).

D.1 info.xml

```
<?xml version="1.0" encoding="utf-8"?>
<fomod>
  <Name>Dawnlight</Name>
  <Author>Patrice</Author>
  <Version>1.2.0</Version>
  <Website>https://www.nexusmods.com/skyrimspedition/mods/98765</Website>
  <Description>Improved lighting, candles and weather for Skyrim SE.</Description>
  <Groups>Visuals and Graphics</Groups>
  <Game>skyrimSpecialEdition</Game>
</fomod>
```

D.2 ModuleConfig.xml

Features covered by the example:

- **Required files:** requiredInstallFiles (a file and a folder).
- **Step 1:** SelectAll group (option Required) and SelectExactlyOne (options Recommended / Optional, res flag).
- **Step 2:** SelectAny group (candles and moon flags) and SelectAtMostOne.
- **Step 3:** visibility condition (<visible>) then SelectAtLeastOne group with a dynamic-type option (dependencyType: Recommended if candles=On, NotUsable if a file is missing) and a CouldBeUsable option.
- **Conditional installs:** two patterns (And and Or operators, flag and file dependencies, Active / Inactive / Missing states).

```
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
  <moduleName>Dawnlight</moduleName>
  <moduleImage path="fomod\header.png"/>
  <requiredInstallFiles>
    <file source="Dawnlight.esp" destination="Dawnlight.esp" priority="0"/>
    <folder source="core\meshes" destination="meshes" priority="0"/>
  </requiredInstallFiles>
  <installSteps order="Explicit">
    <installStep name="Main installation">
      <optionalFileGroups>
        <group name="Core module" type="SelectAll">
          <plugins order="Explicit">
            <plugin name="Base files">
              <description>Scripts and meshes essential to the mod.</description>
              <files>
                <folder source="core\scripts" destination="scripts" priority="0"/>
              </files>
              <typeDescriptor>
                <type name="Required"/>
              </typeDescriptor>
            </plugin>
          </plugins>
        </group>
        <group name="Texture resolution" type="SelectExactlyOne">
          <plugins order="Explicit">
            <plugin name="2K">
              <description>2K textures, good quality/performance balance.</description>
```

```

        <image path="fomod\2k.png"/>
        <conditionFlags>
            <flag name="res">2K</flag>
        </conditionFlags>
        <files>
            <folder source="tex\2k" destination="textures" priority="0"/>
        </files>
        <typeDescriptor>
            <type name="Recommended"/>
        </typeDescriptor>
    </plugin>
    <plugin name="4K">
        <description>4K textures, for powerful setups.</description>
        <image path="fomod\4k.png"/>
        <conditionFlags>
            <flag name="res">4K</flag>
        </conditionFlags>
        <files>
            <folder source="tex\4k" destination="textures" priority="1"/>
        </files>
        <typeDescriptor>
            <type name="Optional"/>
        </typeDescriptor>
    </plugin>
</plugins>
</group>
</optionalFileGroups>
</installStep>
<installStep name="Ambience options">
    <optionalFileGroups>
        <group name="Lighting effects" type="SelectAny">
            <plugins order="Explicit">
                <plugin name="Dynamic candles">
                    <description>Adds candles with flickering light.</description>
                    <conditionFlags>
                        <flag name="candles">On</flag>
                    </conditionFlags>
                    <files>
                        <folder source="opt\candles" destination="meshes" priority="0"/>
                    </files>
                    <typeDescriptor>
                        <type name="Optional"/>
                    </typeDescriptor>
                </plugin>
                <plugin name="Enhanced moon">
                    <description>Increases the intensity of the moonlight.</description>
                    <conditionFlags>
                        <flag name="moon">On</flag>
                    </conditionFlags>
                    <files>
                        <folder source="opt\moon" destination="textures" priority="0"/>
                    </files>
                    <typeDescriptor>
                        <type name="Optional"/>
                    </typeDescriptor>
                </plugin>
            </plugins>
        </group>
        <group name="Sound ambience" type="SelectAtMostOne">
            <plugins order="Explicit">
                <plugin name="Night sounds">
                    <description>Adds a nighttime sound ambience.</description>
                    <files>
                        <folder source="opt\sound" destination="sound" priority="0"/>
                    </files>
                    <typeDescriptor>
                        <type name="Optional"/>
                    </typeDescriptor>
                </plugin>
            </plugins>
        </group>
    </optionalFileGroups>
</installStep>
<installStep name="ENB compatibility">
    <visible>
        <dependencies operator="And">
            <flagDependency flag="res" value="4K"/>
        </dependencies>
    </visible>
</installStep>

```

```

    <fileDependency file="KnownConflict.esp" state="Inactive"/>
  </dependencies>
</visible>
<optionalFileGroups>
  <group name="ENB preset" type="SelectAtLeastOne">
    <plugins order="Explicit">
      <plugin name="Light ENB">
        <description>Low-cost ENB preset.</description>
        <files>
          <file source="enb\light.ini" destination="enblocal.ini" priority="0"/>
        </files>
        <typeDescriptor>
          <type name="Optional"/>
        </typeDescriptor>
      </plugin>
      <plugin name="Ultra ENB">
        <description>High-end ENB preset (dynamic type).</description>
        <files>
          <file source="enb\ultra.ini" destination="enblocal.ini" priority="1"/>
        </files>
        <typeDescriptor>
          <dependencyType>
            <defaultType name="Optional"/>
            <patterns>
              <pattern>
                <dependencies operator="And">
                  <flagDependency flag="candles" value="On"/>
                </dependencies>
                <type name="Recommended"/>
              </pattern>
              <pattern>
                <dependencies operator="Or">
                  <fileDependency file="EnbHelper.dll" state="Missing"/>
                </dependencies>
                <type name="NotUsable"/>
              </pattern>
            </patterns>
          </dependencyType>
        </typeDescriptor>
      </plugin>
      <plugin name="Experimental extension">
        <description>Additional effects, not guaranteed.</description>
        <files>
          <folder source="enb\extra" destination="textures" priority="2"/>
        </files>
        <typeDescriptor>
          <type name="CouldBeUsable"/>
        </typeDescriptor>
      </plugin>
    </plugins>
  </group>
</optionalFileGroups>
</installStep>
</installSteps>
<conditionalFileInstalls>
  <patterns>
    <pattern>
      <dependencies operator="And">
        <flagDependency flag="res" value="4K"/>
        <flagDependency flag="candles" value="On"/>
      </dependencies>
      <files>
        <folder source="cond\candles4k" destination="textures" priority="0"/>
      </files>
    </pattern>
    <pattern>
      <dependencies operator="Or">
        <flagDependency flag="moon" value="On"/>
        <fileDependency file="SkyUI_SE.esp" state="Active"/>
      </dependencies>
      <files>
        <file source="cond\patch.esp" destination="patch.esp" priority="0"/>
      </files>
    </pattern>
  </patterns>
</conditionalFileInstalls>
</config>

```

Appendix E — License and credits

The source code of XIMOD Architect is distributed under the MIT license (© 2025-2026). The third-party components below retain their own license; the full details and attributions appear in the THIRD-PARTY-NOTICES file provided with the application.

Component	Source	License
XIMOD Architect code	—	MIT
Noto fonts	Google Fonts / Noto project	SIL Open Font License 1.1
Flags (SVG)	drapeauxdespays.fr	Royalty-free (commercial use included)
ISO 3166-1 alpha-3 country codes	ISO — iso.org/obp	Free access
ISO 639-3 language codes	SIL — iso639-3.sil.org	Attribution required (see below)
Rust libraries	crates.io ecosystem	MIT / Apache-2.0 (mostly)

ISO 639-3 attribution: the language codes come from iso639-3.sil.org (source to be credited). The data added by XIMOD — endonyms, associated font, country ↔ language mappings — is not part of the ISO 639-3 standard, and the standard’s identifiers are not modified. The application must moreover not provide any means of redistributing the full set of codes.